

Enterprise RAG Intelligence System

Secure Retrieval-Augmented Generation with Role-Based Access Control

Project Type	Full-Stack AI Application — RAG + RBAC
Technology Stack	Node.js, Express, Anthropic Claude API (claude-sonnet-4-6), Vanilla JS, HTML/CSS
Data Sources	7 enterprise source type documents
Access Roles	5 RBAC roles (Finance, Engineering, HR, Executive, Intern)
Submitted By	Mohammed Abdul Shanawaz
Date	June 13, 2026

■ SECTION 1

Project Overview

This project delivers a secure, context-aware Retrieval-Augmented Generation (RAG) system designed for large-scale enterprise environments. The system enables employees to query heterogeneous data sources using natural language, while enforcing strict role-based access control (RBAC) to prevent unauthorized data exposure.

The challenge addressed by this system is one faced by every large organization: critical information is stored across disconnected silos — PDFs, databases, JSON logs, compliance records — making it difficult for employees to get timely, accurate answers without manually searching multiple systems.

Core Capabilities

- Natural language query interface — employees ask questions in plain English
- Intelligent source routing — queries are automatically directed to relevant data sources
- Role-based access enforcement — every response is filtered by the user's permissions
- Grounded AI responses — answers are generated using only retrieved, authorized context
- Real-time pipeline visualization — users see the RAG process steps as they happen
- Live role switching — RBAC behavior can be demonstrated across all 5 roles

■ SECTION 2

System Architecture

The system follows a three-tier architecture: a browser-based frontend, a Node.js/Express proxy server, and the Anthropic Claude API. This separation ensures API keys are never exposed to the browser and all access control decisions are enforced server-side.

Architecture Layers

```
Browser (index.html + app.js)
  |
  | POST /api/chat (with role context + system prompt)
  v
Express Server (server.js)  <-- API key lives here only
  |
  | POST https://api.anthropic.com/v1/messages
  v
Anthropic Claude API (claude-sonnet-4-6)
```

RAG Pipeline Steps

Each query passes through 5 sequential steps before a response is generated:

- 1. Query intent analysis — the system identifies the topic and data domain of the question
- 2. Source routing — relevant data sources are identified using keyword matching against 7 source categories
- 3. RBAC permission check — accessible sources are filtered against the current user's role
- 4. Context retrieval — the system assembles the authorized context for the query
- 5. Response generation — Claude generates a grounded, source-cited answer

■ SECTION 3

Enterprise Data Sources

The system connects to 7 enterprise data source categories totalling over 52,000 documents, records, and log entries. Each source has a defined access policy that is enforced at query time.

Data Source	Volume	Accessible By	Content Types
Financial Records	14,200 docs	Finance Manager, Executive	PDFs, spreadsheets, budget reports
Technical Docs	8,700 pages	Engineer, Executive	API specs, deployment guides, architecture
JSON Audit Logs	2.1M entries	Engineer, Executive	System events, errors, traces
HR Records	4,300 records	HR Director, Executive	Employee data, requisitions, org charts
Compliance Docs	892 files	HR Director, Finance, Executive	Policies, certifications, risk assessments

Business Reports	3,600 reports	All roles	KPIs, summaries, dashboards
Operational Data	28,900 rows	Engineer, Executive	Workflows, pipelines, capacity

■ SECTION 4

Role-Based Access Control

RBAC is enforced at two levels: the system prompt sent to Claude includes explicit instructions listing which sources are accessible and which are restricted for the current user. Additionally, the frontend prevents queries from being routed to inaccessible sources before they even reach the API.

Role	Accessible Sources	Typical Queries
Finance Manager	Financial Records, Compliance Docs, Business Reports	Revenue, budget, compliance queries
Senior Engineer	Technical Docs, JSON Audit Logs, Operational Data	Errors, latency, deployment queries
HR Director	HR Records, Compliance Docs, Business Reports	Headcount, training, requisition queries
C-Suite Executive	All 7 sources (full access)	Any enterprise query
Intern	Business Reports only	Public summaries and overviews only

Access Denial Behavior

When a query requires data from a restricted source and no accessible source can answer it, the system returns an explicit access-denied message. This message names the restricted source and instructs the user to contact their administrator — no partial data is leaked.

■ SECTION 5

Project File Structure

```
enterprise-rag/
├── .env           ─ API key and port config (never committed to Git)
├── .gitignore     ─ Excludes .env from version control
├── package.json   ─ Node.js project config with type: module
├── server.js      ─ Express proxy server + static file serving
└── index.html     ─ Complete single-file frontend (HTML + CSS + JS)
```

Key Design Decisions

- Single-file frontend — all HTML, CSS, and JS in one index.html eliminates path resolution issues when served locally

- Proxy server pattern — server.js keeps the Anthropic API key server-side; the browser only calls /api/chat
- ES Module syntax — package.json uses type: module enabling native import/export in Node.js
- Role context in system prompt — RBAC is enforced by injecting accessible/restricted source lists into every Claude API call

■ SECTION 6

Setup & Installation

Prerequisites

- Node.js v18 or higher — download from nodejs.org
- An Anthropic API key — obtain from console.anthropic.com
- A terminal / command prompt

Step-by-Step Installation

6. Create project folder and navigate into it

```
mkdir enterprise-rag
cd enterprise-rag
```

7. Initialize Node.js project and install dependencies

```
npm init -y
npm install express cors dotenv
```

8. Create the environment file with your API key

```
# Windows CMD:
echo ANTHROPIC_API_KEY=sk-ant-your-key-here > .env
echo PORT=3000 >> .env

# Mac/Linux:
echo "ANTHROPIC_API_KEY=sk-ant-your-key-here" > .env
echo "PORT=3000" >> .env
```

9. Update package.json to enable ES modules

```
Add "type": "module" to package.json
```

10. Create server.js and index.html with the provided code

11. Start the server

```
node server.js
```

12. Open browser at http://localhost:3000

■ SECTION 7

Testing the System

RBAC Verification

To verify that role-based access control is working correctly, perform the following tests:

13. Select Finance Manager role — ask about audit logs or HR records — expect access denied
14. Select C-Suite Executive role — ask the same question — expect a full response citing those sources
15. Select Intern role — ask about revenue figures — expect access denied
16. Select Engineer role — ask about critical errors — expect a technical response from JSON Audit Logs

Sample Queries by Role

```
Finance Manager: 'Summarize Q3 revenue performance'
Engineer:        'Show me all critical errors from the past 24 hours'
HR Director:     'What is our current headcount by department?'
Executive:       'Give me a full business health summary'
Intern:          'What public reports are available?'
```

■ SECTION 8

Conclusion

This project demonstrates a production-grade architecture for enterprise AI assistants. The key contributions are:

- A working RAG pipeline that routes queries to relevant data sources based on keyword analysis
- A robust RBAC implementation enforced at both the prompt level and the routing level
- A clean, professional UI with real-time pipeline visualization and role switching
- A secure proxy architecture that keeps API credentials server-side

The system is extensible — real document ingestion, vector embeddings, and a database-backed permission store can be added without changing the core architecture. The foundation built here follows the same patterns used in production enterprise AI deployments.

■ SECTION 9

Result And Execution.

EnterpriseRAG
Intelligence Platform

JD Jane Doe
Finance Manager

Finance

SWITCH ROLE TO EXPLORE RBAC

Finance Manager (Jane Doe)

Finance Manager (Jane Doe)

Senior Engineer (Alex Chen)

HR Director (Sam Rivera)

C-Suite Executive (Morgan Lee)

Intern (Casey Kim)

JSON Audit Logs
2.1M entries

HR Records
4.3K records

Compliance Docs
892 files

Business Reports
3.6K reports

Operational Data
28.9K rows

21K+
docs

3/7
sources

0
queries

RAG Intelligence Query

Live

RBAC active: Jane Doe cannot access Technical Docs, JSON Audit Logs...

Ask anything across your data

Queries are intelligently routed to relevant sources and filtered by your access level.

Summarize Q3 revenue performance

What compliance issues were flagged last quarter?

Show budget variance for engineering

Ask a question across enterprise data...

EnterpriseRAG
Intelligence Platform

JD Jane Doe
Finance Manager

Finance

SWITCH ROLE TO EXPLORE RBAC

Finance Manager (Jane Doe)

Finance Manager (Jane Doe)

Senior Engineer (Alex Chen)

HR Director (Sam Rivera)

C-Suite Executive (Morgan Lee)

Intern (Casey Kim)

Financial Records
14.2K docs

Technical Docs
8.7K pages

JSON Audit Logs
2.1M entries

HR Records
4.3K records

Compliance Docs
892 files

Business Reports
3.6K reports

Operational Data
28.9K rows

21K+
docs

3/7
sources

1
queries

RAG Intelligence Query

Live

RBAC active: Jane Doe cannot access Technical Docs, JSON Audit Logs...

Summarize Q3 revenue performance

Processing...

Analyzing query intent

Routing to data sources

Checking RBAC permissions

Retrieving context

Generating response

Summarize Q3 revenue performance

What compliance issues were flagged last quarter?

Show budget variance for engineering

Ask a question across enterprise data...

EnterpriseRAG
Intelligence Platform

AC Alex Chen
Senior Engineer Engineering

SWITCH ROLE TO EXPLORE RBAC
Senior Engineer (Alex Chen)

DATA SOURCES

Financial Records
14.2K docs

Technical Docs
8.7K pages

JSON Audit Logs
2.1M entries

HR Records
4.3K records

Compliance Docs
652 files

Business Reports
3.6K reports

Operational Data
28.9K rows

21K+
docs

3/7
sources

2
queries

RAG Intelligence Query

Live

RBAC active: Alex Chen cannot access Financial Records, HR Records...

Summarize Q3 revenue performance

RAG Intelligence - Jane Doe

No response generated.

Financial Records

Processing...

Analyzing query intent

Routing to data sources

Checking RBAC permissions

Retrieving context

Generating response

Show critical errors from the past 24 hours

Show critical errors from the past 24 hours

What API endpoints have the highest latency?

Summarize the latest deployment report

Ask a question across enterprise data...